

Architettura dei calcolatori e sistemi operativi

Lucidi tratti dal **CAP. IO1: Input Output a Livello Hardware**
delle dispense:

Programmazione e Struttura del Sistema Operativo Linux

Prof. Giuseppe Pelagatti, Politecnico di Milano

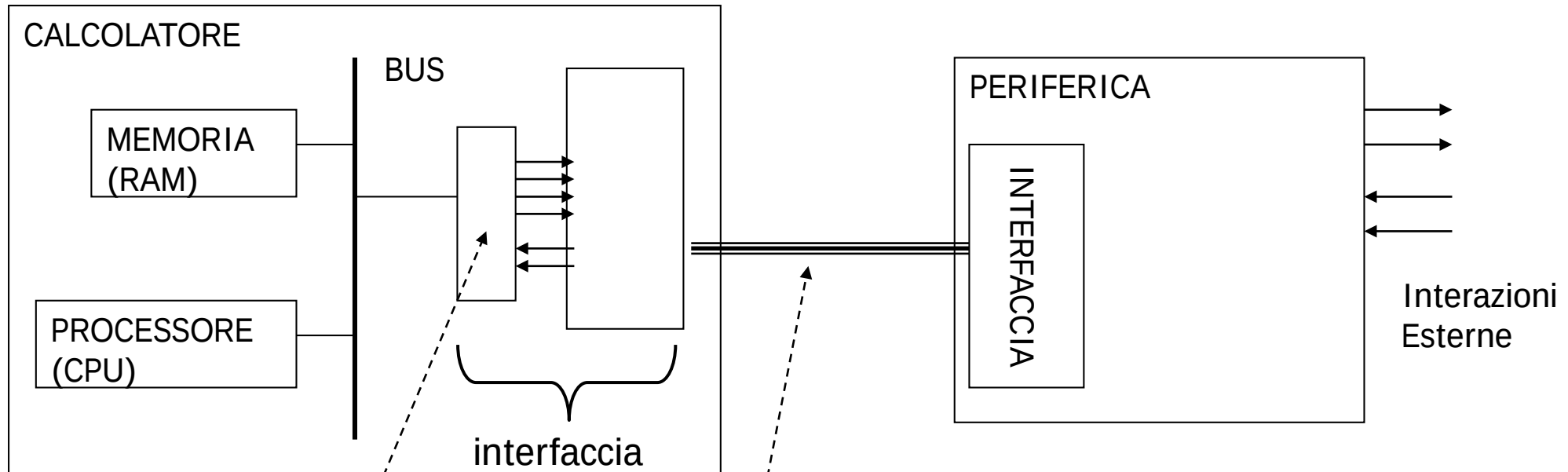


Accesso alle periferiche (1)

- Le unità periferiche interagiscono con il calcolatore, e cioè con il processore e la memoria centrale, attraverso **interfacce di ingresso/uscita** (dette anche **adattatori**) collegate al bus di sistema.
 - Le interfacce sono quindi dispositivi circuitali che consentono al calcolatore di scambiare informazioni con le unità periferiche.
 - Per ogni coppia (*tipo-di-CPU*, *tipo-di-Periferica*) sono presenti adattatori dedicati cioè una coppia di interfacce: lato-CPU e lato-periferica
 - La struttura delle interfacce di ingresso/uscita è costituita, in modo molto schematico
 - da alcuni **registri** leggibili e/o scrivibili da parte del processore, ma comunicanti anche con il mondo esterno (periferiche), e
 - da circuiti ausiliari.



Collegamento di una periferica al calcolatore (a livello di sistema)



Registri di I/O **indirizzabili**
-stato
-comandi
-dati
(trattati come bit in registri)

BUS DI INTERFACCIAMENTO:

Seriale/parallelo
Dedicato/condiviso
Standard/ad hoc

...

Esempi:

RS232C, IEEE488 (a.k.a. GPIB), USB, SCSI, IEEE1394 (aka FireWire)
stato/comandi/dati trattati come segnali o codici



Registri di interfaccia di una periferica

- Considerando uno schema molto semplificato, una interfaccia è costituita dai seguenti registri
 - **Registro dati periferica**
 - contiene i dati che la periferica deve leggere (periferica di uscita) o ha scritto (periferica di ingresso)
 - **Registro comandi (o controllo) periferica**
 - contiene indicazioni sulle operazioni che la periferica deve compiere e sul suo modo di funzionamento
 - **Registro di stato periferica**
 - contiene informazioni sullo stato di funzionamento della periferica (ad esempio “pronta” a gestire un nuovo dato, o “occupata”)
 - bit **Ready**: relativo alla possibilità, per il processore, di svolgere un’operazione di lettura o scrittura di un dato
 - una periferica di ingresso, come una tastiera, è in stato di pronto quando un tasto è stato premuto e quindi esiste un carattere nel suo registro dati che il processore può leggere
 - una periferica di uscita, come una stampante, è pronta se può ricevere un dato dal processore per stamparlo.



Esempi di funzionamento

- Una **stampante** molto rudimentale può funzionare in modo semplificato come segue:
 - appena “accesa” Ready=1, registro dati vuoto
 - quando la CPU scrive un dato nel registro dati Ready va a 0
 - quando la stampante ha finito di stampare il dato e il registro dati è nuovamente vuoto, Ready torna a 1
- Una **tastiera** può funzionare in modo semplificato come segue:
 - appena “accesa” Ready=0, registro dati vuoto
 - quando viene premuto un tasto, Ready va a 1
 - quando la CPU legge il dato, Ready torna a 0



Istruzioni di Input-Output

- ❑ **Memory Mapped I/O**: i registri delle periferiche sono associati ad indirizzi omogenei con quelli della memoria centrale
 - la CPU accede ai registri con le stesse istruzioni di accesso alla memoria
 - la memoria non deve rispondere agli indirizzi che mappano l'I/O

- ❑ **Port Mapped I/O**: sono disponibili istruzioni specifiche per l'accesso ai registri di periferica
 - lo spazio degli indirizzi delle periferiche è separato da quello della memoria. Gli indirizzi dei registri delle periferiche vengono chiamati Port
 - le istruzioni specifiche sono del tipo
 - **IN** port_di_input (legge dal registro associato a port_di_input)
 - **OUT** port_di_output (scrive nel registro associato a port_di_output)

Il **MIPS** non ha istruzioni specifiche per I/O e quindi utilizza il Memory Mapped I/O (hp di lavoro: primi 256 byte riservati)



Tecniche di gestione dell'input/output

- La **gestione dell'I/O** può avvenire tramite:
 1. **controllo di programma**
 2. **Meccanismo di interruzione**
 3. **Accesso Diretto alla Memoria (DMA – *Direct Memory Access*)**

- Le tecniche di gestione devono consentire **interazioni** tra calcolatore e periferica per svolgere:
 - la **sincronizzazione** tra periferica e calcolatore;
 - il **trasferimento** del dato da periferica a calcolatore o viceversa.



Sincronizzazione e trasferimento: esempi

□ **Stampa di un carattere su stampante: interazioni**

- la stampante deve segnalare al calcolatore che è disponibile (pronta) ad accettare un nuovo dato per la stampa (*sincronizzazione*);
- il calcolatore esegue un'istruzione di uscita **OUT** sul registro dati della stampante, che potrà quindi eseguirne la stampa (*trasferimento*).

□ **Acquisizione di un carattere da tastiera: interazioni**

- la tastiera deve segnalare al calcolatore che è stato premuto un tasto e quindi esiste un carattere disponibile (*sincronizzazione*);
- il calcolatore esegue un'istruzione di ingresso **IN** dal registro dati della tastiera per acquisire il valore del carattere (*trasferimento*).

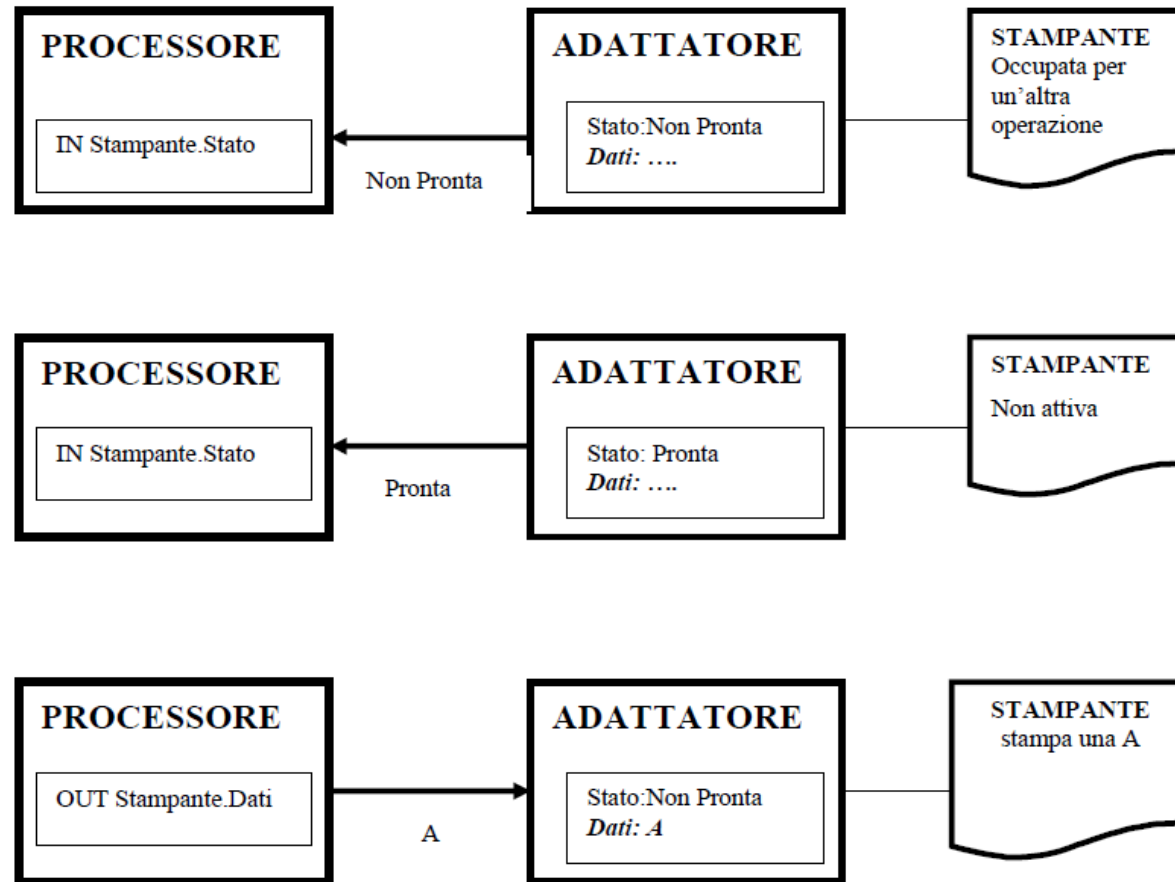


Gestione dell'I/O a controllo di programma (1)

- ❑ La sincronizzazione e il trasferimento vengono eseguiti dal programma di gestione della periferica (**device driver**)
- ❑ Il programma di gestione della periferica:
 - **legge il registro di stato** dell'interfaccia della periferica (**IN registro_di_stato**)
 - se lo stato è "periferica non pronta" (bit *Ready*=0) il programma attende che la periferica sia pronta e torna a leggere il registro di stato; altrimenti
 - **esegue il trasferimento** del dato tramite:
 - **IN_registro_dati** se la periferica è di ingresso (ad es. lettura dato da tastiera)
 - **OUT_registro_dati** se la periferica è di uscita (ad es. scrittura dato verso stampante)



Esempio: Stampante a controllo di programma



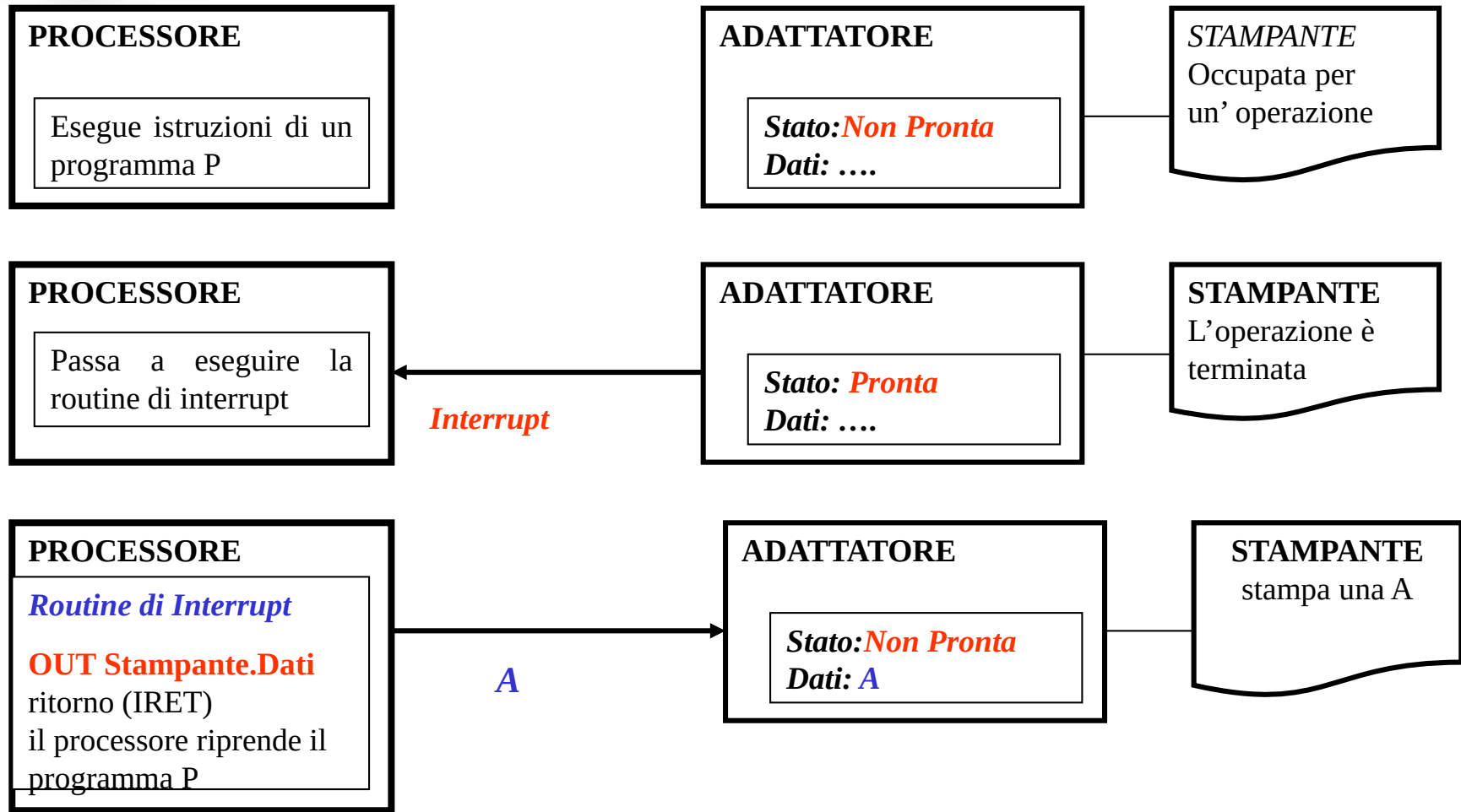


Dal controllo di programma al meccanismo di interrupt

- ❑ Problema: La gestione delle periferiche a **controllo di programma** implica da parte del processore un **ciclo di attesa di periferica pronta**.
 - ❑ Nel Sistema Operativo invece vogliamo che il processore ponga il processo in attesa e passi ad eseguire altre attività fino a quando la periferica non diventa pronta
 - ❑ Questo obiettivo viene ottenuto utilizzando il **meccanismo di interrupt**:
 - richiede che una periferica segnali tramite un apposito segnale di **interrupt** il passaggio da occupata a pronta
 - Il segnale di interrupt segnala la transizione del bit *Ready* da 0 a 1 nel *registro di stato* della periferica
-



Esempio: stampante gestita da meccanismo di interrupt





Gestione dell'I/O basato su interrupt (1)

Il meccanismo di interruzione (**interrupt**) si basa su:

- un insieme di **eventi** rilevati a **livello hardware** dal processore (**interruzioni**). Per es. un segnale opportuno di una periferica, una condizione di errore ...
- Un insieme di **funzioni**, ognuna associata in generale ad un evento. Ogni funzione viene chiamata **gestore dell'interrupt o routine di interrupt**
- Quando il processore rileva un evento:
 - **interrompe il programma** in esecuzione
 - **passa automaticamente a eseguire la routine di interrupt** corrispondente, ed esegue il servizio richiesto
 - al termine dell'esecuzione della routine, **torna ad eseguire il programma** che era stato interrotto (in modo **trasparente** rispetto al programma stesso)



Esempio: tastiera con gestione a interrupt

Acquisizione di un carattere da tastiera con gestione a interrupt

- ❑ Quando l'interfaccia della periferica scrive il dato nel registro dati, con un **segnale di interruzione allerta il processore (sincronizzazione)**.
- ❑ Il processore **interrompe** l'esecuzione del programma in corso e salta automaticamente a eseguire la parte di programma che legge il registro dell'interfaccia (**trasferimento**).
- ❑ Al termine di questo, il processore riprende il programma interrotto.



Gestione dell'I/O basato su interrupt (2)

Il meccanismo di interruzione è **simile** a quello di invocazione di una funzione e infatti prevede:

- la cessione del controllo dell'esecuzione tra il programma interrotto e la routine di risposta all'interrupt.
 - E' quindi necessario **salvare sullo stack l'indirizzo di ritorno** di tale programma (e cioè l'indirizzo dell'istruzione successiva a quella in cui il programma è stato interrotto)
- il ritorno all'esecuzione del programma interrotto.
 - Le routine di risposta all'interrupt utilizzano, per il ritorno, una istruzione macchina **IRET speciale** e diversa dalla RET dei programmi utente.

La **differenza fondamentale** rispetto alle chiamate a funzione è che queste sono attivate dal programma in esecuzione, mentre le routine di interrupt sono “scatenate” da eventi rilevati dal processore e asincroni rispetto all'esecuzione del programma che viene interrotto

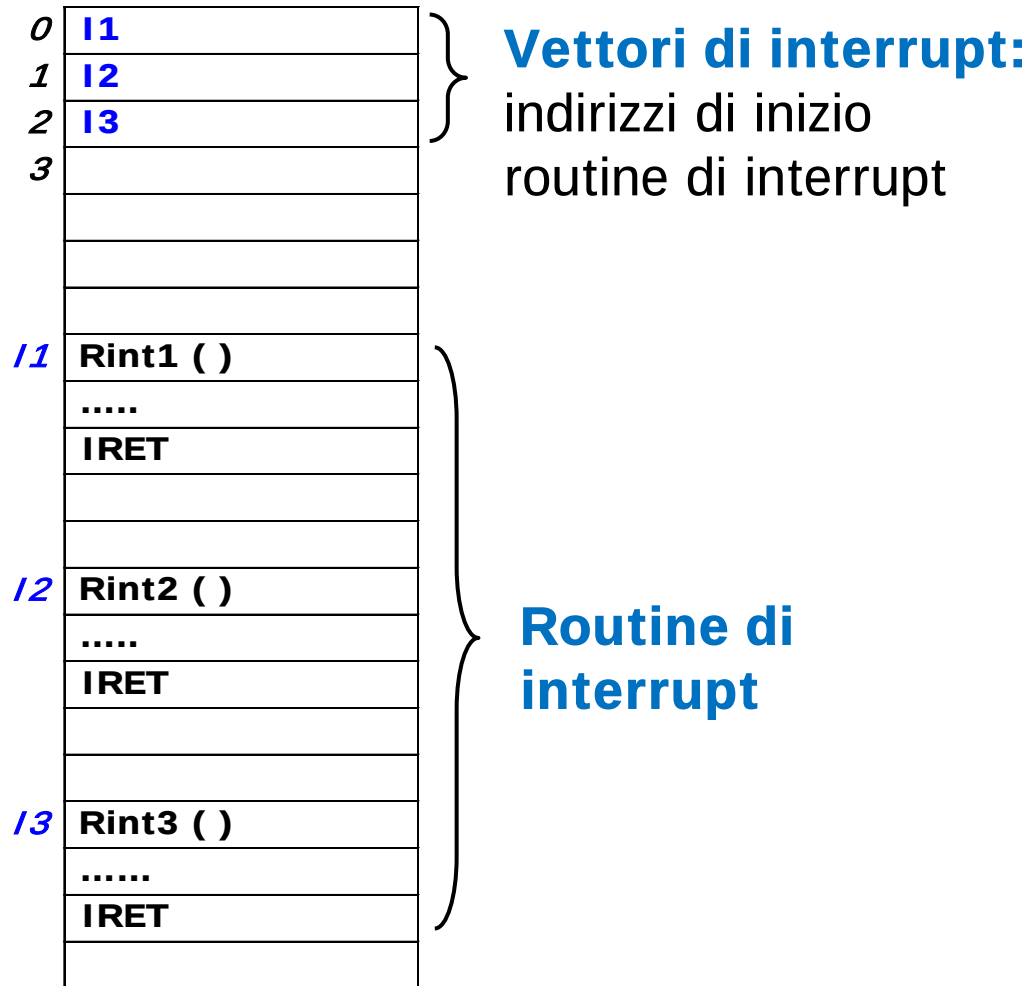


Interrupt annidati

- ❑ Il salvataggio dell'indirizzo di ritorno sullo stack consente di gestire **interruzioni annidate**, al pari di quanto accade nella gestione delle funzioni nidificate.
- ❑ Un interrupt può quindi interrompere, in linea di principio, la routine di risposta di un altro interrupt (**interrupt annidati**) in base a livelli di priorità degli interrupt



Vettori e Routine di Interrupt





Accesso Diretto alla Memoria - DMA

- Il meccanismo di accesso diretto alla memoria (**DMA – Direct Memory Access**) da parte di periferiche prevede che:
 - Il driver della periferica trasferisca in modo **autonomo**, cioè **senza l'intervento del processore**, un certo numero di dati (cioè un blocco alla volta... non un byte alla volta) in memoria centrale o dalla memoria centrale
 - Il meccanismo di DMA è utilizzato da periferiche quali i dischi magnetici dotate di interfacce “intelligenti” (DMA e controllori del disco) che sono in grado di trasferire velocemente uno o più settori di disco da o in memoria centrale
- Il **DMA Controller (o Adattatore DMA)** effettua il trasferimento dati a blocchi tra periferiche e memoria autonomamente, cioè senza l'intervento da parte del processore. Il controllore del DMA è quindi in grado di **generare i segnali per accedere a memoria** (indirizzo, dati, controllo)



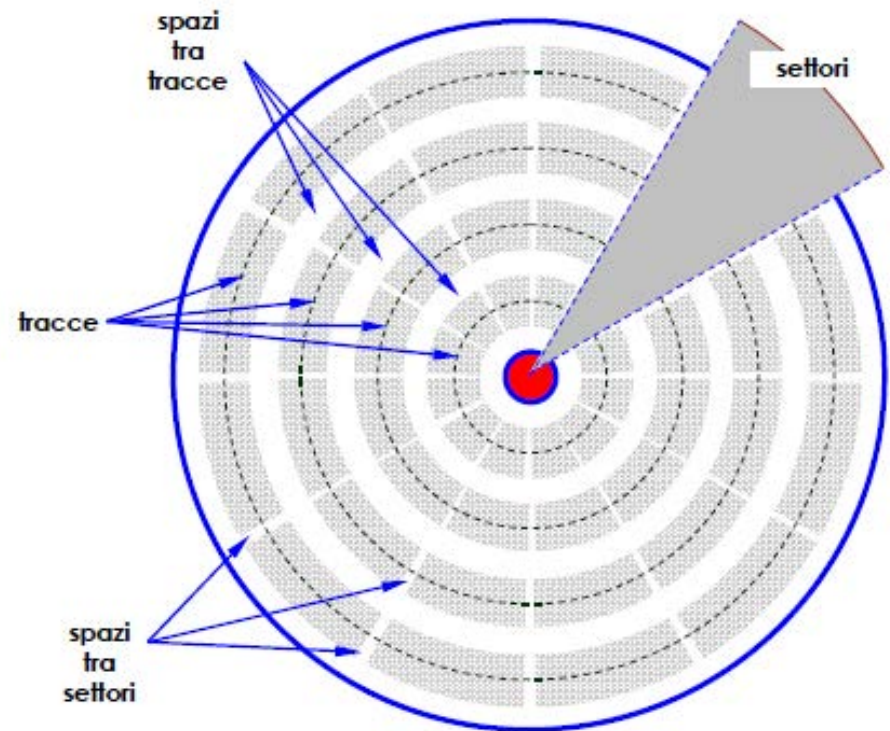
Dispositivi di memorizzazione non-volatile

- ❑ Le interfacce dei dispositivi di memorizzazione non-volatile **Hard Disk Drivers (HDD)** e **Solid State Drive (SSD)** utilizzano la tecnica di gestione ad accesso diretto alla memoria (**DMA**) per il trasferimento **fisico** dei dati
- ❑ Dal punto di vista logico (File System) l'indirizzamento dei dati si basa sullo schema di indirizzamento **LBA (Logical Block Address)** nel quale l'intero disco è rappresentato come un vettore lineare di blocchi
- ❑ Nel seguito useremo il termine **Volume** per indicare qualsiasi dispositivo di memorizzazione non volatile di massa, siano essi **HDD** oppure **SSD**, dotato di uno schema di indirizzamento **LBA**.
- ❑ A livello logico il **blocco** costituisce l'unità fondamentale di informazione che viene trasferita con una sola operazione tra il disco e la memoria centrale



Dispositivi di memorizzazione non-volatile (2)

- A livello fisico su HHD, i dati sono memorizzati su disco in cerchi concentrici detti **tracce** e le tracce sono suddivise in **settori**.
- Il **settore** (o più d'uno) è l'**unità** che viene **trasferita** con una sola operazione di accesso tra il disco e la memoria centrale (**il blocco è quindi un multiplo del settore**).
- L'operazione di trasferimento è controllata dall'interfaccia del disco





DMA (1)

La tecnica **DMA** prevede le seguenti fasi:

- 1) Predisposizione.** Una procedura (del Sistema Operativo) scrive nei registri dell'interfaccia (o adattatore) delle periferica:
- l'indirizzo della **memoria** dal quale iniziare il trasferimento
 - l'indirizzo sulla **periferica** dal quale iniziare il trasferimento
 - il **numero di dati** (parole di memoria) da trasferire
 - la **direzione** del trasferimento (lettura da periferica o scrittura su periferica)
- 2) Attivazione.** La procedura del Sistema Operativo scrive nel registro di controllo dell'interfaccia il comando di avviamento **"start"** dell'operazione e quindi il processore passa ad eseguire altri programmi. La periferica può a questo punto procedere in modo **autonomo** nel trasferimento.



DMA (2)

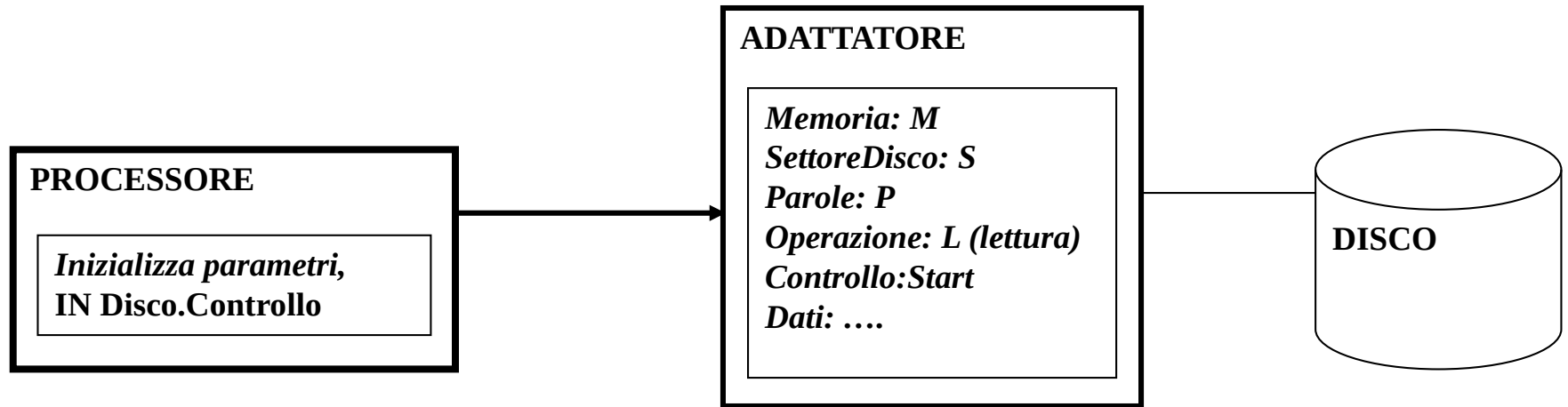
3) *Trasferimento di un blocco di dati.*

- Nel caso di **lettura** da periferica, ogni volta che dalla periferica arriva un dato all'interfaccia, l'interfaccia scrive il dato in memoria, incrementa l'indirizzo in memoria, decrementa il numero di dati ancora attesi e, se non sono arrivati tutti, aspetta il prossimo dato
- Nel caso di **scrittura** su periferica, ogni volta che la periferica è disponibile ad accettare nuovi dati, l'interfaccia preleva il dato da memoria, incrementa l'indirizzo in memoria, decrementa il numero dei dati ancora da trasferire e, se non sono trasferiti tutti, aspetta che la periferica sia disponibile per il prossimo dato

4) Conclusione. Se l'ultimo dato è stato trasferito, l'interfaccia segnala al processore tramite **interrupt** che l'operazione in DMA si è conclusa

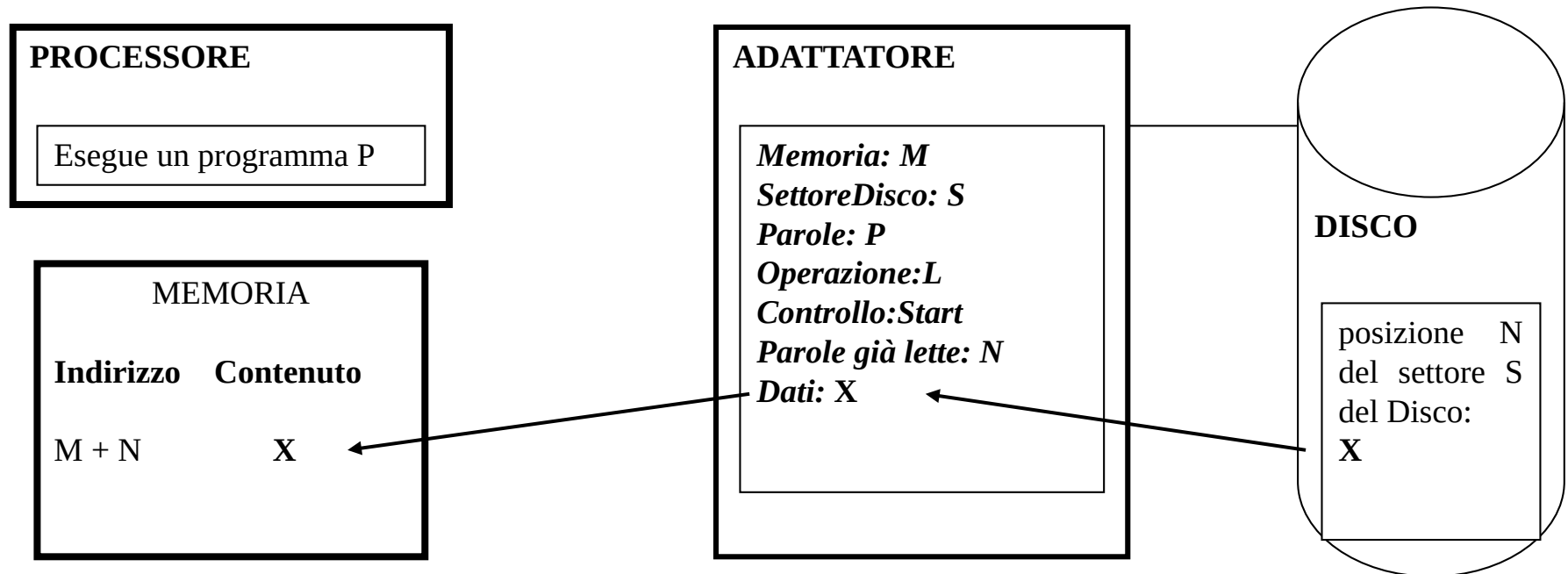


DMA – Esempio di operazione di lettura da disco: il processore inizializza l'adattatore del disco



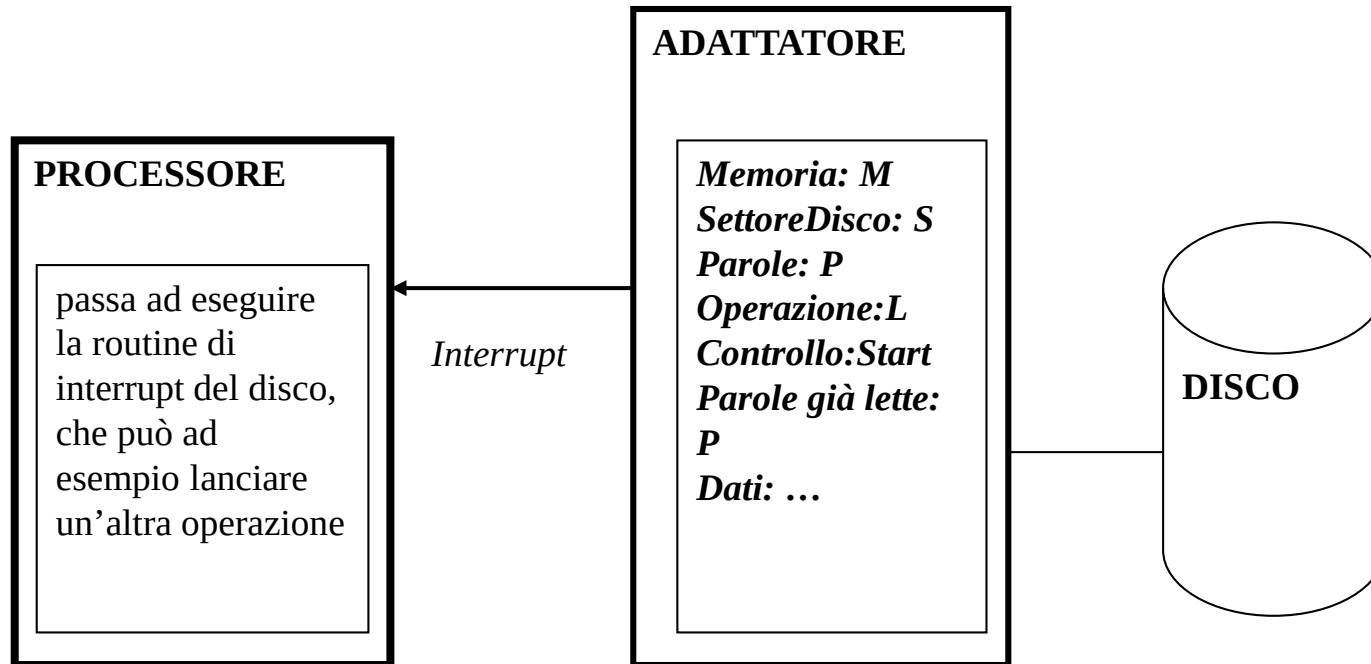


DMA – Esempio di operazione di lettura da disco: N parole già trasferite da disco





DMA – Esempio: L'adattatore del disco genera un interrupt alla fine della lettura di P parole





DMA e Sistema Operativo

- Ai fini della realizzazione del Sistema Operativo, le periferiche gestite in **DMA** richiedono che il Sistema Operativo gestisca opportunamente delle aree di memoria (dette *buffer*) per il trasferimento dati in modalità DMA da/per le periferiche.
- Inoltre, Linux nella definizione dei **device driver** distingue tra:
 - **Block devices**: periferiche a blocchi in DMA, tipicamente i dischi (caratterizzate da accesso casuale o diretto)
 - **Character devices**: periferiche a caratteri, tipicamente tastiera e stampante (caratterizzate da accesso sequenziale)

Il bus del calcolatore



Il bus del calcolatore

- Le unità funzionali di un calcolatore (processore, coprocessori, memoria, unità di interfacciamento alle periferiche) sono interconnesse tramite un organo di collegamento: il **BUS**
 - Bus interni sono confinati dentro una singola unità funzionale
 - Bus esterni, solitamente standardizzati collegano tra loro le unità funzionali
- Una **transazione di bus (o ciclo di bus)**: è un insieme di operazioni sul bus che permette di raggiungere un obiettivo
 - **Transazione di trasferimento**: trasferisce una certa quantità di informazione tra due *endpoints*
 - **Transazione di interrupt**: protocollo di segnalazione di un *interrupt* alla CPU e accettazione, (predisposizione dell'attivazione della routine di risposta all'interrupt)
- la quantità di informazione trasferita da un singolo ciclo di bus varia in genere da 8 a 64 bit

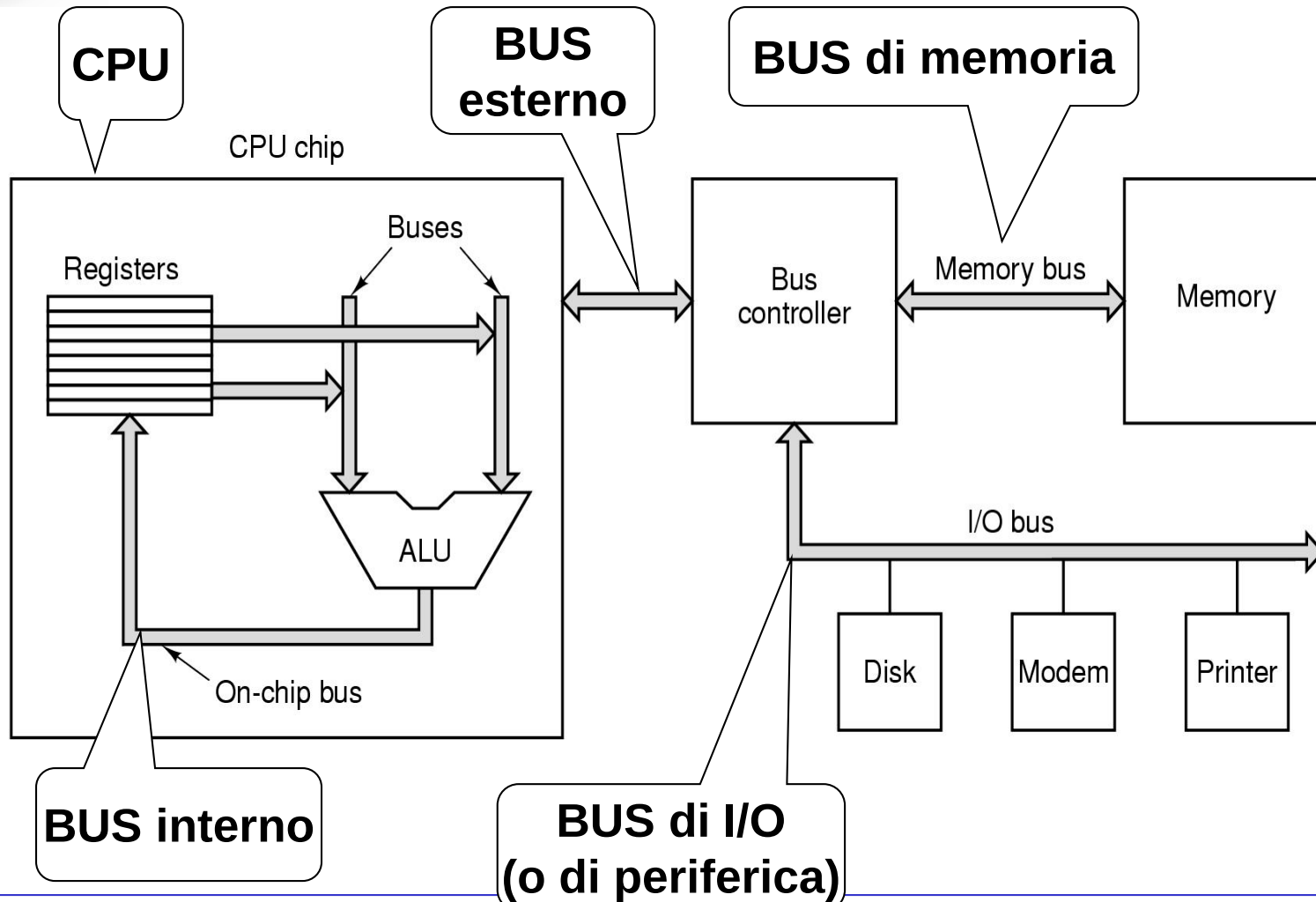


Bus interni ed esterni

- I **BUS** del calcolatore si distinguono in:
 - **BUS interni**, confinati all'interno d'una singola unità funzionale, i quali collegano i blocchi funzionali contenuti nell'unità;
 - **BUS esterni**, che collegano le unità funzionali; sono solitamente standardizzati (es. PCI - Peripheral computer interconnect)
 - I calcolatori più semplici sono dotati di **un solo BUS esterno (BUS di sistema)** che collega CPU, memoria e unità di I/O
 - La maggior parte dei calcolatori odierni è dotata di **numerosi BUS esterni**, in particolare:
 - **BUS di memoria**, collegante la CPU e le unità funzionali di memoria (banchi di memoria);
 - **BUS di I/O**, collegante la CPU e le unità funzionali di I/O
-

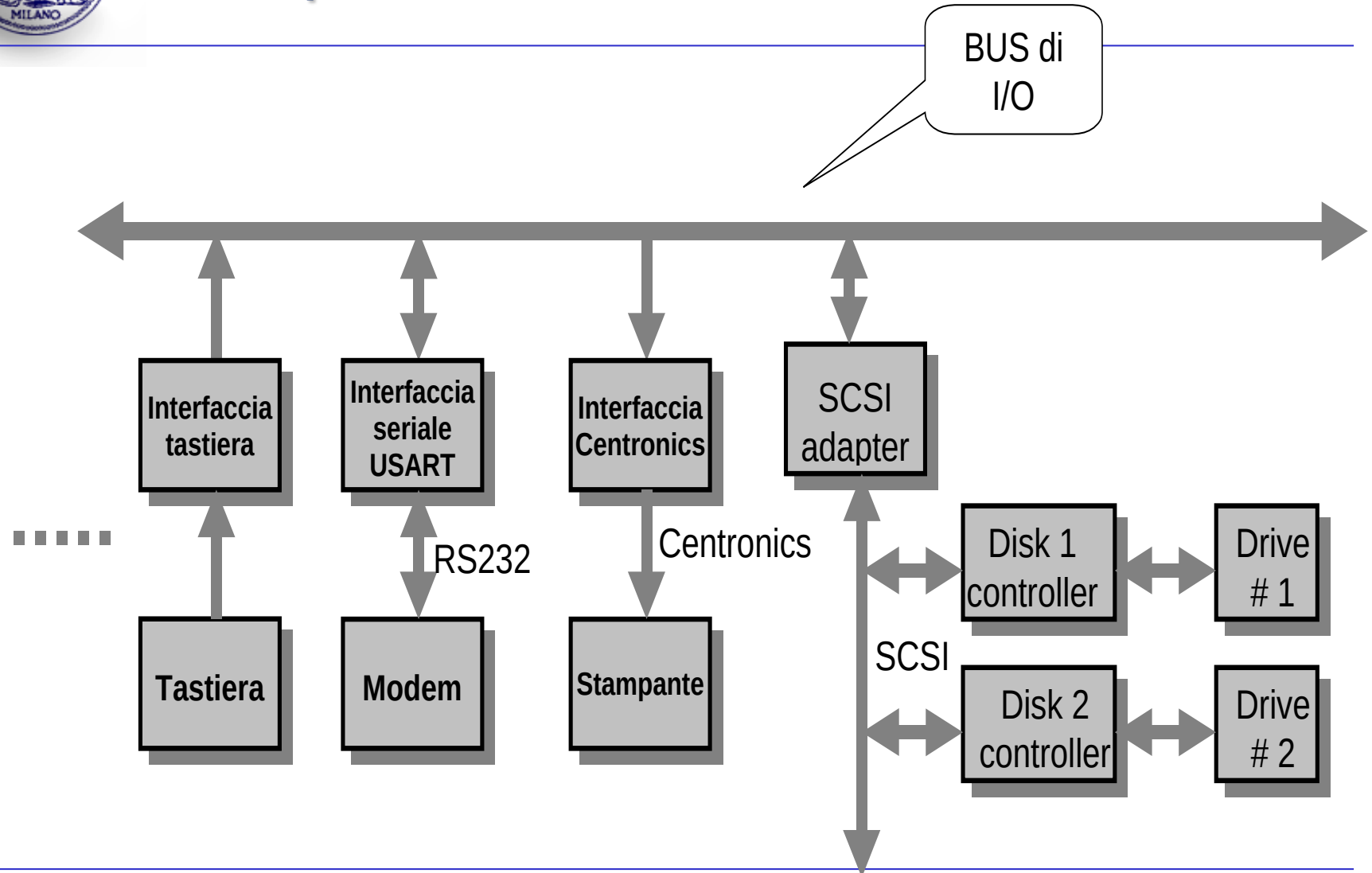


Sistema con diversi BUS





Esempi di interfacce del BUS di I/O



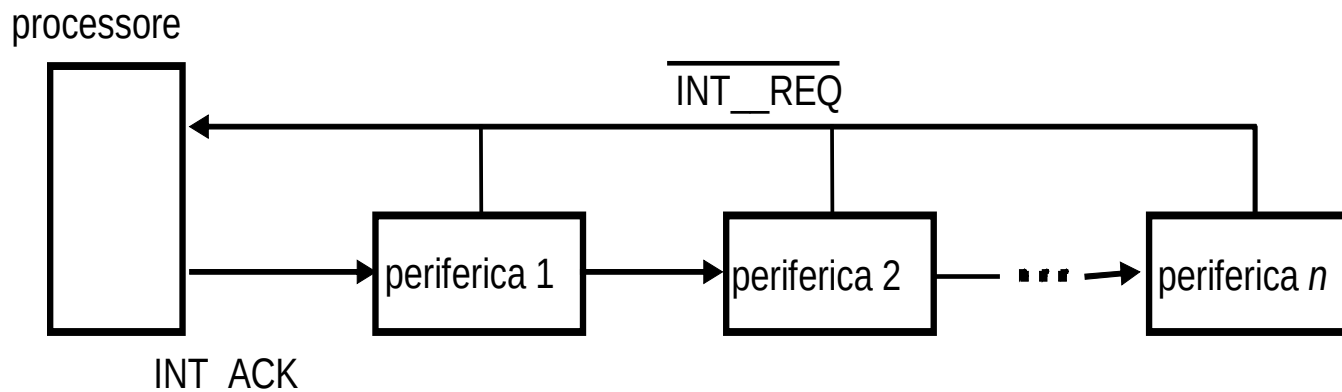


Transazione di interrupt

E' l'insieme di operazioni che conduce da una segnalazione di interrupt da parte di una periferica alla sua presa in carico da parte della CPU

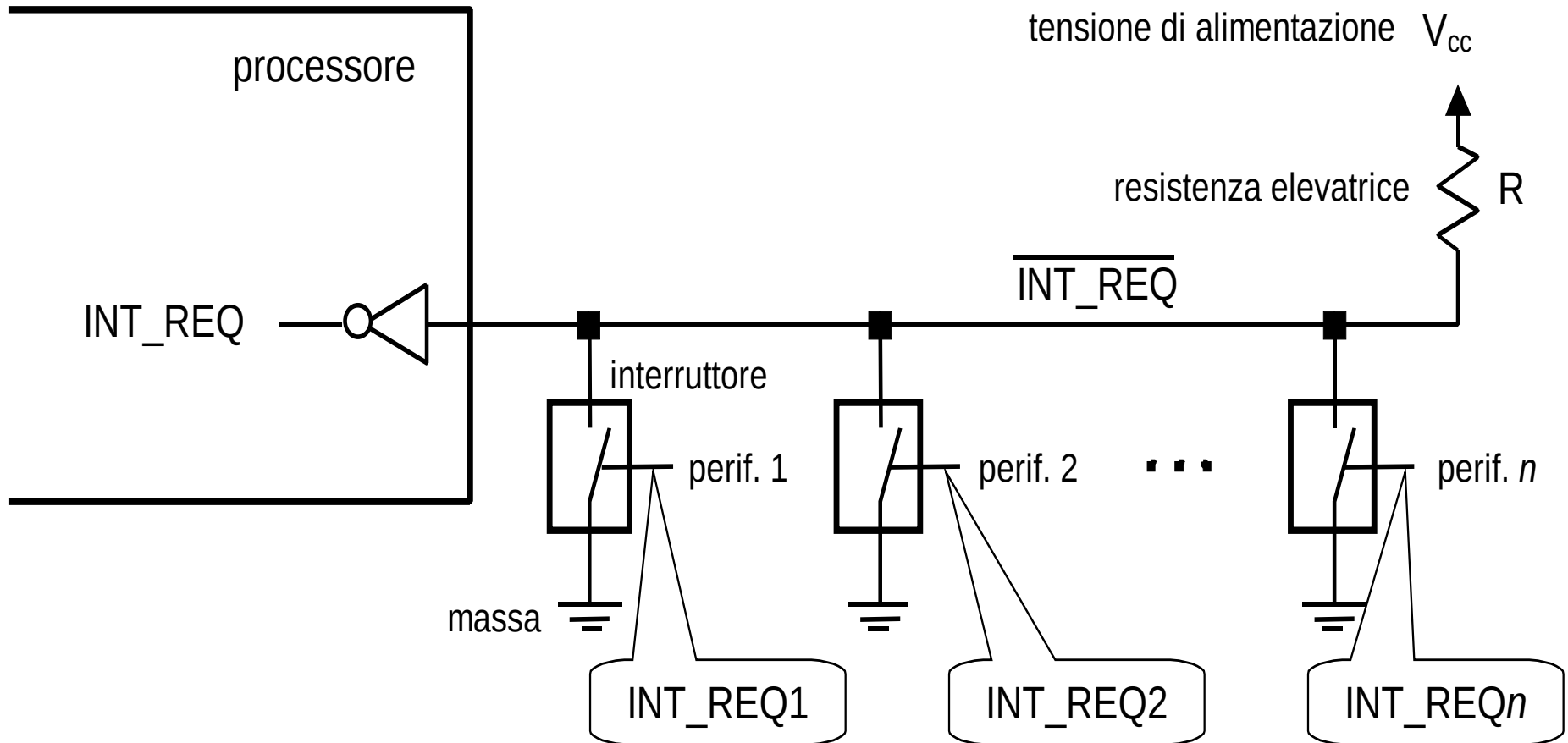
Esistono diversi modi di collegare le linee di interrupt delle periferiche alla CPU; ad esempio:

- **INT_REQ** (richiesta di interrupt) in **wired_or**
- **INT_ACK** (accettazione dell'interrupt) in **daisy chain** (cioè a ghirlanda o festone) per la propagazione del segnale
 - in corrispondenza della generazione del segnale di **INT_ACK** può partire un ciclo di trasferimento (lettura) in cui il processore acquisisce dalla periferica interessata **il vettore di interrupt**





Collegamento WIRED OR di una linea di INT_REQ (segnale attivo basso)





Transazione di trasferimento

- Una **transazione di trasferimento** si può generalmente scomporre in due fasi principali:
 - fase di **arbitraggio**: serve a selezionare un'unità, detta **MASTER**, che controlla il bus durante l'operazione
 - fase di **trasferimento** vero e proprio, durante la quale avvengono le seguenti operazioni
 - il **MASTER** seleziona un'altra unità, detta **SLAVE**, con la quale operare
 - il **MASTER** indica la direzione del trasferimento:
 - lettura (dallo SLAVE verso il MASTER)
 - scrittura (dal MASTER verso lo SLAVE)
 - viene effettuato il vero e proprio trasferimento di unità di informazione tra MASTER e SLAVE
- Le unità che svolgono il ruolo di **Master** e di **Slave** sono fissate per ogni singola operazione di trasferimento del bus, ma possono variare tra operazioni diverse



Unità funzionali che possono diventare MASTER

- Nei sistemi multiprocessore le **diverse CPU** possono tutte diventare **MASTER**
- In determinate circostanze anche altre unità funzionali possono assumere il ruolo di **MASTER**

Master	Slave	Esempio
CPU	Memoria	Prelievo istruzioni e lettura/scrittura dati
CPU	Unità di I/O	Ricezione/invio dati da/a un'unità di I/O
CPU	Coprocessore	La CPU dà istruzioni al coprocessore
I/O	Memoria	Accesso diretto alla memoria (DMA)*
Coprocessore	CPU	Il coprocessore legge operandi dalla CPU

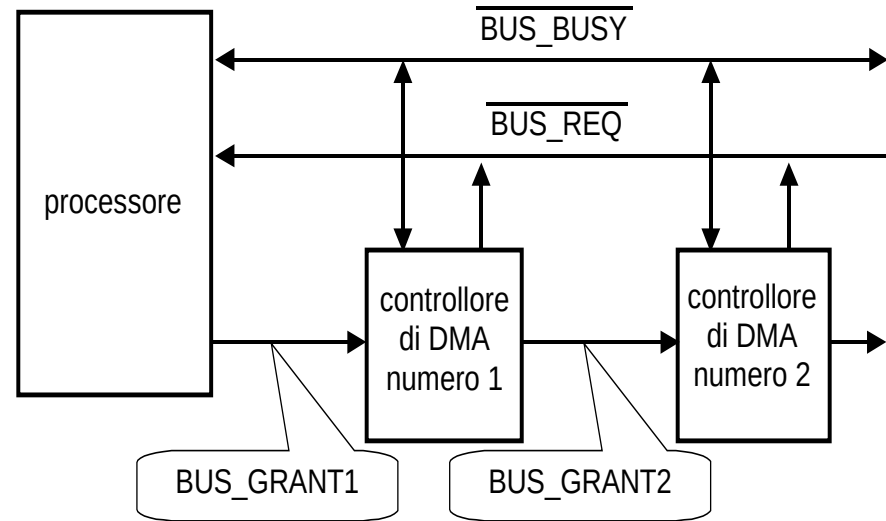


Arbitraggio centralizzato in Daisy Chain

- fase di arbitraggio per il **prossimo MASTER** sovrapposta alla fase di trasferimento controllata dal **MASTER corrente**
- questo meccanismo di arbitraggio prevede:
 - un'unità funzionale apposita, che svolge la funzione di **arbitro del BUS**; talvolta la funzione di arbitro è svolta dalla CPU
 - alcune linee che collegano l'arbitro alle unità funzionali potenziali richiedenti il controllo del BUS:
 - **Bus Request** (richiesta di cessione del controllo) in **wired or**
 - **Bus Busy** (indicazione che esiste un Master corrente che sta eseguendo un trasferimento) in **wired or**
 - **Bus Grant** (conferma di cessione del controllo) in **daisy chain**
- Il segnale **BUS_BUSY** asserito dall'unità Master fino a quando ha terminato di usare il bus per un trasferimento per indicare che il bus è occupato.
 - Dato che questa linea è leggibile da tutte le unità, l'unità che diventa Prossimo Master può controllarla e attendere che il bus diventi libero prima di iniziare a utilizzarlo per il proprio trasferimento.



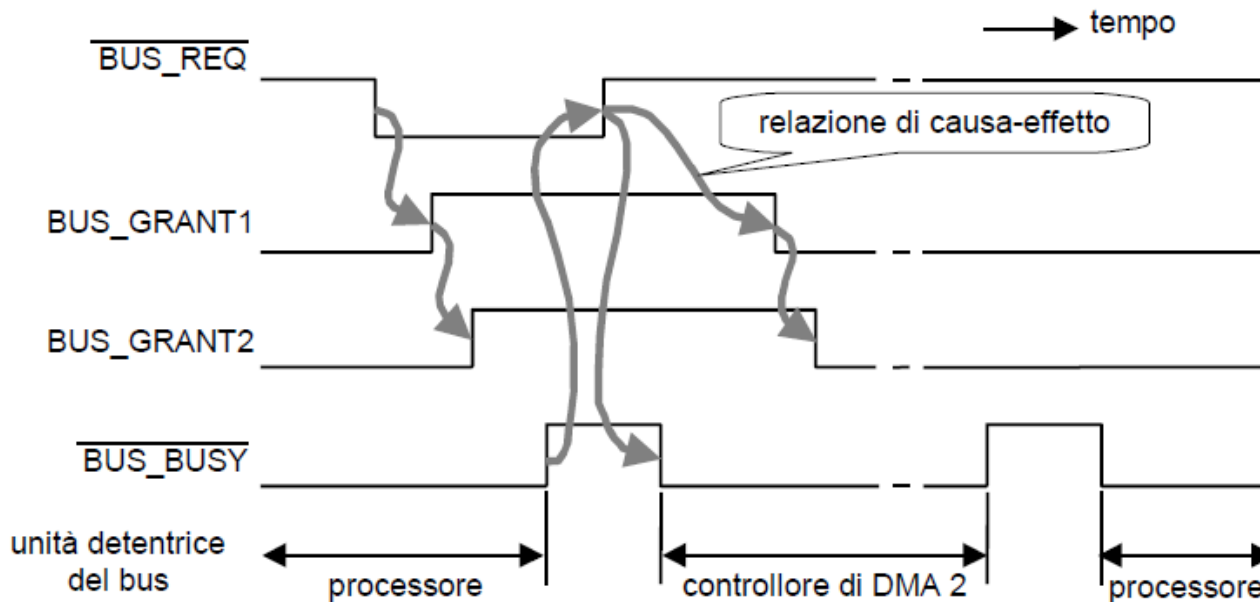
Arbitraggio centralizzato tra CPU (arbitro) e diversi controllori DMA con collegamento a festone semplice



- ❑ Quando l'unità di arbitraggio riceve una richiesta di BUS (BUS_REQ), attiva la linea di conferma (BUS_GRANT)
- ❑ La conferma viene passata in cascata alle unità potenziali richiedenti:
 - – se un'unità non ha una richiesta pendente, passa la conferma all'unità successiva
 - – altrimenti, trattiene per sé la conferma, senza passarla all'unità successiva, e prende il controllo del BUS, comportandosi come MASTER
- ❑ L'unità che in un dato istante detiene il controllo del bus lo comunica attivando BUS_BUSY



Arbitraggio centralizzato tra CPU (arbitro) e diversi controllori DMA con collegamento a festone semplice



- ❑ **L'unità 2 richiede il bus**
- ❑ Segnali di **BUS_REQ** e **BUS_BUSY** **attivi bassi**
- ❑ **BUS_BUSY** tenuto a 0 dal master per tutto l'intervallo di tempo in cui detiene il bus
- ❑ **BUS_REQ** può essere attivato anche se **BUS_BUSY** attivo e viene disattivato dall'unità richiedente non appena ottiene il bus



Linee del Bus

- Riassumendo, un bus basato sui meccanismi descritti deve possedere le seguenti linee:
 - **INT_REQ e INT_ACK**
 - **BUS_BUSY, BUS_REQ e BUS_GRANT**
 - **RW** (assumiamo convenzionalmente RW=1 lettura, RW=0 scrittura)
 - **linee di indirizzo**
 - **linee dati**



Temporizzazione del Bus

- ❑ Per realizzare la scansione delle **transazioni (cicli) di BUS** esistono due metodi fondamentali:
- ❑ BUS **sincrono**: funzionamento governato da un segnale di clock (*bus clock*)
- ❑ BUS **asincrono**: funzionamento governato solo dall'interazione tra master e slave

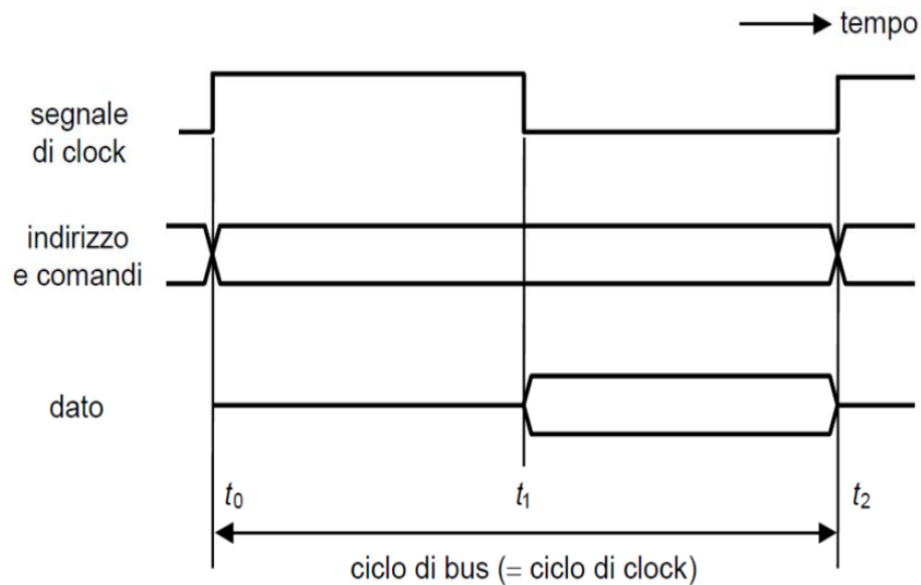


Bus sincrono

- ❑ Il BUS di controllo contiene una linea per il segnale di *bus clock*, a frequenza prestabilita
 - ❑ Il clock viene distribuito a tutte le unità funzionali collegate al BUS sulle cui transizioni avvengono gli eventi
 - ❑ Il segnale di clock scandisce le varie transizioni di segnale e il passaggio da un ciclo di BUS al ciclo successivo
 - ❑ Tutte le unità sanno sempre quando si passa al ciclo successivo
 - ❑ La durata del ciclo di bus dipende dai dispositivi e dai tempi di latenza/propagazione
 - ❑ **Un bus sincrono opera velocemente se collegato a poche unità funzionali con ritardi di propagazione omogenei (ad es. memoria)**
-



Ciclo di Lettura del Bus sincrono

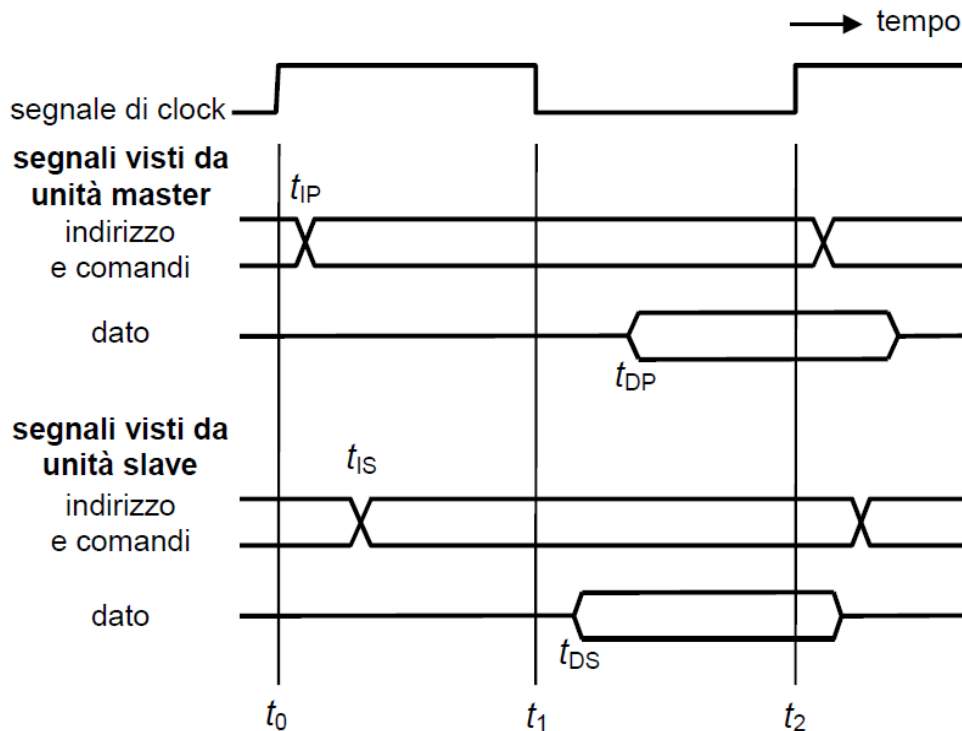


- $t_1 - t_0$ non deve essere inferiore al massimo ritardo di propagazione tra due unità collegate al bus incluso il tempo necessario per decodificare l'indirizzo e interpretare la richiesta sul bus di controllo in modo che lo slave sia in grado di rispondere a partire da t_1
- $t_2 - t_1$ non deve essere inferiore alla somma del massimo tempo di propagazione dei dati sul bus dati e del tempo di setup del registro
- A t_2 il dato sul bus viene campionato dalla CPU e memorizzato in un registro

La frequenza di completamento di un ciclo di bus è pari a 2 ritardi di propagazione



Ciclo di Lettura del Bus sincrono: dettagli



- $t_1 - t_0$ non deve essere inferiore al massimo ritardo di propagazione tra due unità collegate al bus incluso il tempo necessario per decodificare l'indirizzo e interpretare la richiesta sul bus di controllo in modo che lo slave sia in grado di rispondere a partire da t_1
- $t_2 - t_1$ non deve essere inferiore alla somma del massimo tempo di propagazione dei dati sul bus dati e del tempo di setup del registro
- A t_2 il dato sul bus viene campionato dalla CPU e memorizzato in un registro



Bus asincrono

- ❑ Non esiste alcun segnale di clock comune alle unità funzionali collegate al BUS del calcolatore
- ❑ Le transizioni di segnale e il passaggio da un ciclo di BUS al ciclo successivo non sono sincronizzati
- ❑ **Le unità funzionali osservano il BUS di controllo:** quando avviene una transizione di segnale significa che si verifica un avanzamento dell'operazione
 - Il bus è dotato di opportuni segnali di controllo (MSYN, SSYN, di cui vengono osservate le transizioni) che realizzano un protocollo di sincronizzazione a segnale (ad es. full-handshake)

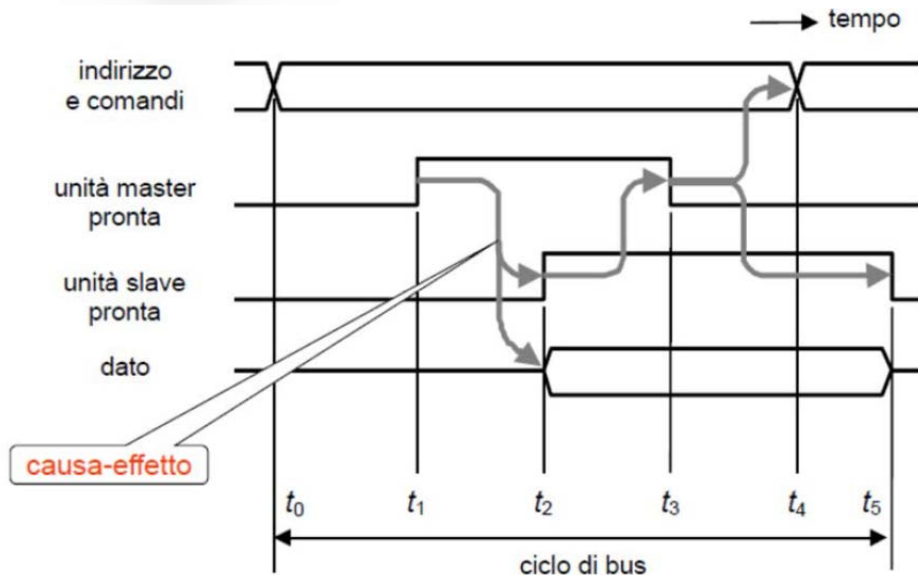


Segnali di controllo per l'handshake

- ❑ **MSYN** (Master Synchronisation): unità master pronta
 - il MASTER segnala di avere impostato indirizzo e RW;
- ❑ **SSYN** (Slave Synchronisation): unità slave pronta
 - lo SLAVE segnala di avere completato l'operazione di trasferimento
- ❑ Poiché non esiste un segnale di clock che marchi gli istanti di tempo in cui i vari segnali di controllo si possono attivare o disattivare, nel **diagramma temporale** vanno indicati i **rapporti causa-effetto** esistenti tra i vari segnali di controllo
- ❑ I segnali di controllo transitano di valore in reazione a transizioni precedenti



Operazione di Lettura sul bus asincrono



t4: Il **MASTER** toglie l'indirizzo e comandi dal bus

t5: lo **SLAVE** disattiva SSYN

Il tempo di propagazione di SSYN verso il master deve essere atteso per poter ricominciare con un nuovo ciclo

La frequenza di completamento di un ciclo di bus è pari a 4 ritardi di propagazione

□ t0: Il **MASTER** (CPU) manda l'indirizzo sul BUS indirizzi, attiva RW

□ t1: dopo averli tutti stabilizzati, il master
attiva MSYN

Il tempo t1-t0 compensa lo sfasamento temporale dovuto al fatto che i segnali sul bus si propagano con ritardi variabili in funzione della distanza

□ t2: Lo **SLAVE** (memoria) esegue l'operazione, nel minor tempo possibile, e fornisce la parola sul BUS dati, poi **attiva SSYN**

t2-t1 varia in base alle unità coinvolte e deve tenere conto del ritardo con cui le unità slave emettono il dato sul bus e il ritardo di propagazione sul bus

□ t3: segnale SSYN arriva al master e rende noto che il dato è presente sul bus

t3-t2: Unità master prima di leggere il dato dal bus DATI deve attendere i ritardi di propagazione del dato + tempo di setup del registro

t3: Il **MASTER** disattiva MSYN

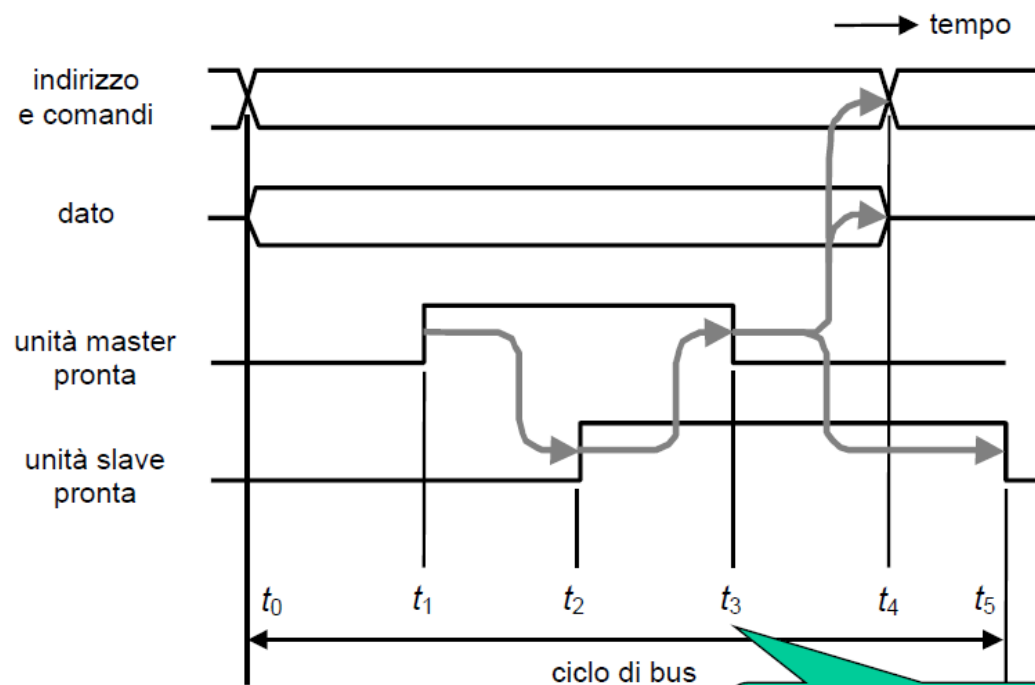


Procedura full-handshake

- ❑ Lo schema asincrono appena visto è incentrato sulla seguente procedura:
 - - MSYN viene attivato
 - - SSYN viene attivato in risposta a MSYN
 - - MSYN viene disattivato in risposta a SSYN
 - - SSYN viene disattivato in risposta a MSYN
- ❑ La procedura ha nome **full-handshake**: è indipendente dal clock ed è insensibile alla presenza di SLAVE lenti
 - Fully interlocked: ogniqualevolta uno dei due segnali di sincronismo varia, anche l'altro varia in risposta



Operazione di scrittura sul bus asincrono



Lo SLAVE ha campionato il dato sul bus e memorizzato internamente